

Matplotlib Visualization and the Capstone Workflow

These chapters belong together because the Capstone project carries NumPy, Pandas, and Matplotlib through one realistic data workflow. The plotting API is not the hard part: first decide **what relationship you need to see**, then select the chart that expresses it. Choosing the chart matters more than memorizing method names.

- Python 3 - Matplotlib
- tutor style
- visual explanations
- FAQ included
- 8 chart types + integrated workflow

00 Choose the Relationship Before the Chart

What relationship do you need to see?	Recommended tool
One sequence across time or order	<code>ax.plot()</code>
Raw values against a smoothed signal	two <code>ax.plot()</code> calls + <code>ax.legend()</code>
One value compared across categories	<code>ax.bar()</code>
Two-variable relationship with highlighted groups	call <code>ax.scatter()</code> once per group
Several related views at once	<code>plt.subplots(2, 1)</code>
Overall distribution of one numeric variable	<code>ax.hist()</code>
Compare distributions across categories	<code>ax.boxplot()</code> with a list of arrays
Overview of pairwise correlations	<code>ax.imshow(corr_matrix)</code>

01 - START
Fixed skeleton
fig, ax, save, close

02
Dual line
labels + legend

03
Bar
Series mapping

04
Scatter
split groups

05
Panels
axes array

06
Missing values
judge structure

07
Z-score
transfer formula

08
Histogram
one distribution

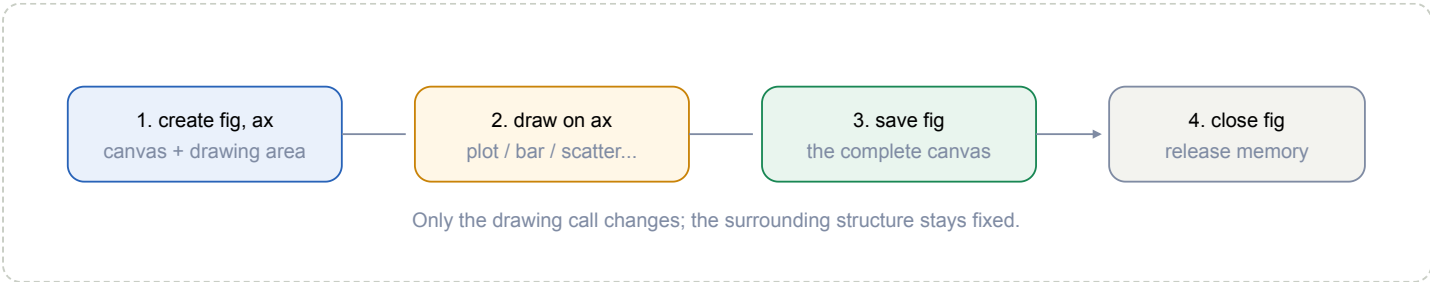
09
Boxplot
group distributions

10
Heatmap
matrix overview

The Capstone lesson is judgment, not new syntax. Sections 06 and 07 ask whether rows are independent records or one continuous signal, and whether an old formula can migrate into a new container. You already know `fillna`, `groupby`, rolling windows, and Z-scores; the task is choosing and combining them correctly.

01 The Fixed Four-Step Plotting Skeleton

- One workflow, regardless of chart type



```
fig, ax = plt.subplots()           # Create the canvas and drawing area
ax.plot(signal.values)             # Draw on ax
ax.set_title(...)                 # Add title and axis labels
fig.savefig(output_path)          # Save the whole figure
plt.close(fig)                    # Close and release memory
```

`signal.values` turns a Pandas Series into an ordinary numeric array. With only y-values, Matplotlib supplies x positions `0, 1, 2, ...`. Keep the order fixed: **draw on ax -> decorate -> save fig -> close fig**.

• Template 1 - line chart

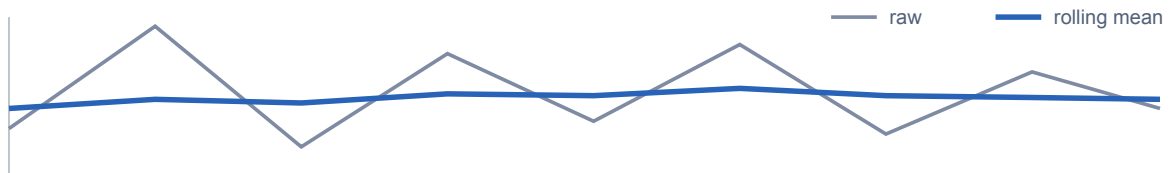
```
def plot_line(series, output_path, title="", xlabel="", ylabel=""):
    fig, ax = plt.subplots()
    ax.plot(series.values)
    ax.set_title(title)
    ax.set_xlabel(xlabel)
    ax.set_ylabel(ylabel)
    fig.savefig(output_path)
    plt.close(fig)
```

Q. What are `fig` and `ax` responsible for?

`fig` is the entire canvas and controls saving, size, and multi-panel layout. `ax` is one concrete drawing region and owns the lines, points, title, and axis labels. Draw on `ax`; save and close `fig`.

02

Two Lines and a Legend: Raw vs Smoothed



Two plot calls share the same axes and scale.

```
ax.plot(signal.values, label="raw")
ax.plot(signal.rolling(window=window).mean().values,
        label="rolling mean")
ax.legend()
```

Each `label=` names one plotted object, but the names become visible only after `ax.legend()`. A useful comparison keeps both lines on the same scale so the smoothed signal can be read against the original positions.

• Template 2 - raw and smoothed

```
def plot_raw_vs_smoothed(series, output_path, window=3):
    fig, ax = plt.subplots()
    ax.plot(series.values, label="raw")
    ax.plot(series.rolling(window=window).mean().values,
            label="rolling mean")
    ax.legend()
    fig.savefig(output_path)
    plt.close(fig)
```

Draw lines without labels, then call `ax.legend()`

The legend only lists plotted objects that have a label.

Add `label=` to every line.

03 Bar Charts from a Pandas Series

• Map Series structure directly to the chart

averages



The Series index supplies category labels; its values supply bar heights.

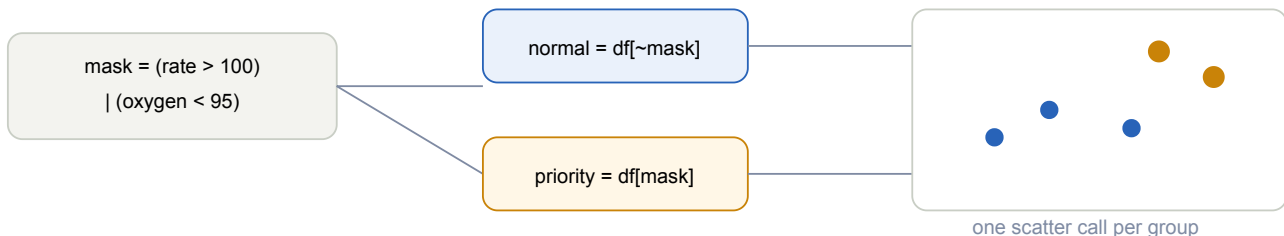
Visualize a groupby result

bar

```
averages = dataframe.groupby("department")["heart_rate"].mean()
ax.bar(averages.index, averages.values)
# index -> category names on the x-axis
# values -> numeric bar heights
```

Use a bar chart whenever the result has the shape **category -> one numeric value**. In practice, this is often the direct visualization of a Pandas `groupby(...).mean()` result.

04 Grouped Scatter Plots: Split First, Draw Twice



Boolean split followed by separate drawing calls

scatter

```
mask = (dataframe["heart_rate"] > 100) | \
        (dataframe["oxygen_level"] < 95)
normal = dataframe[~mask]
priority = dataframe[mask]

ax.scatter(normal["heart_rate"], normal["oxygen_level"],
           label="normal")
ax.scatter(priority["heart_rate"], priority["oxygen_level"],
           label="priority")
ax.legend()
```

A scatter plot does not infer groups by itself. Split the DataFrame with a Boolean mask, then call `scatter` for each group. Matplotlib advances its default color cycle on each call, giving the groups distinct colors without a hand-built color list.

• Template 3 - grouped scatter

Reusable two-group scatter function

grouped

```
def plot_grouped_scatter(dataframe, mask, x_col, y_col, output_path):
    group_a, group_b = dataframe[~mask], dataframe[mask]
    fig, ax = plt.subplots()
    ax.scatter(group_a[x_col], group_a[y_col], label="normal")
    ax.scatter(group_b[x_col], group_b[y_col], label="priority")
    ax.legend()
    fig.savefig(output_path)
    plt.close(fig)
```

Incorrect

Why it fails

Correct

```
ax.scatter(x, y,
label=mask)
```

label names the whole call; it does not switch color for each point.

Split first, then call **scatter** once per group.

Q. Why do the scatter groups change color automatically?

Every plotting call advances Matplotlib's default color cycle. That is why separate group calls are readable and scalable: the grouping lives in the data split, while color assignment remains automatic.

05 Multiple Panels with plt.subplots(2, 1)

axes[0] - Heart rate



axes[1] - Variability



2 rows x 1 column -> axes is an array of two drawing regions

Raw signal above, rolling variability below

subplots

```
fig, axes = plt.subplots(2, 1)
axes[0].plot(signal.values)
axes[0].set_title("Heart rate")
axes[1].plot(signal.rolling(window=window).std())
axes[1].set_title("Variability")
fig.tight_layout()
fig.savefig(output_path)
```

plt.subplots(rows, columns) returns an array of axes when more than one panel is requested, so select regions with **axes[i]**. Call **tight_layout()** before saving to prevent titles and axis labels from colliding.

• Template 4 - two stacked panels

Reusable two-panel function

panels

```
def plot_two_panels(series, output_path, window=3):
    fig, axes = plt.subplots(2, 1)
    axes[0].plot(series.values)
    axes[0].set_title("Raw")
    axes[1].plot(series.rolling(window=window).std())
    axes[1].set_title("Variability")
    fig.tight_layout()
    fig.savefig(output_path)
    plt.close(fig)
```

06 Capstone Judgment: Which Missing-Value Method?

Do adjacent rows represent the same entity across an ordered sequence?

No: independent patient rows
`fillna(median)`

Yes: continuous signal
`interpolate / ffill / bfill`

12 patients measured once each

Median imputation for independent records

`fillna`

```
copied_dataframe = dataframe.copy()
median_value = dataframe["oxygen_level"].median()
copied_dataframe["oxygen_level"] = (
    copied_dataframe["oxygen_level"].fillna(median_value)
)
```

Read the data-generating structure before choosing the function. Twelve patients measured once are independent records, not one evolving signal. Interpolation and forward/backward fill assume meaningful order or neighborhood; adjacent patients do not have that relationship.

Incorrect

Why it fails

Correct

`interpolate()` across independent patient rows

Interpolation assumes that neighboring rows are meaningfully ordered.

`fillna(median_value)`

07 Z-Score Detection Migrates from NumPy to Pandas

Pandas Series
`df["heart_rate"]`

`.to_numpy()`
ordinary array

same formula
 $z = (x - \text{mean}) / \text{std}$

The container changes; the mathematics does not.

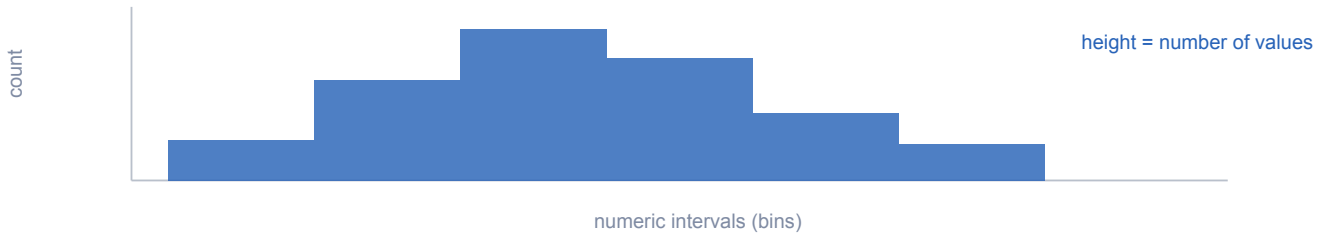
Filter rows whose absolute Z-score stays under the threshold

`z-score`

```
array = copied_dataframe["heart_rate"].to_numpy()
mean = array.mean()
std = array.std()
z_score = (array - mean) / std
mask = np.abs(z_score) <= threshold
return copied_dataframe[mask]
```

This is the NumPy Z-score formula from an earlier chapter, now applied to a DataFrame column. Recognizing that the same method works after a simple `.to_numpy()` conversion is the central transfer skill.

08 Histogram: The Overall Shape of One Distribution



Count observations in six numeric intervals

hist

```
ax.hist(dataframe["heart_rate"], bins=6)
ax.set_title("Heart rate distribution")
```

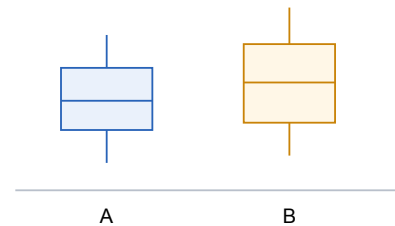
bins sets the number of intervals or their exact edges. Unlike **describe()**, which summarizes a variable with mean, standard deviation, and quantiles, a histogram reveals shape: concentration, skew, multiple peaks, and unusually distant values.

09 Boxplots: Compare Distributions by Category

departments = [A, B]

groups[0] = df[dept == A]["hr"]

groups[1] = df[dept == B]["hr"]



boxplot expects a list whose elements are the one-dimensional arrays for each group.

Build one numeric vector per department

boxplot

```
departments = dataframe["department"].unique()
groups = [
    dataframe[dataframe["department"] == department]["heart_rate"]
    for department in departments
]
ax.boxplot(groups, tick_labels=departments)
```

A boxplot needs a **list of separated groups**, not one full column. Use a list comprehension to extract the value column once per category. Choose this chart when group means are not enough and you need medians, spread, and possible outliers.

• Template 5 - boxplot by group

Reusable grouped-distribution function

groups

```
def plot_boxplot_by_group(dataframe, group_col, value_col, output_path):
    labels = dataframe[group_col].unique()
    groups = [dataframe[dataframe[group_col] == g][value_col] for g in labels]
    fig, ax = plt.subplots()
    ax.boxplot(groups, tick_labels=labels)
    fig.savefig(output_path)
    plt.close(fig)
```

Incorrect

Why it fails

Correct

```
ax.boxplot(dataframe["heart_rate"])
```

It produces only one box, so there is no category comparison.

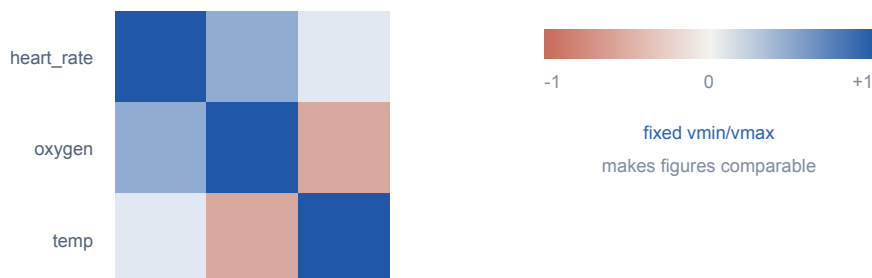
Build a **groups** list with one array per category.

Q. What is the difference between a histogram and a bar chart?

A bar chart maps named, discrete categories to one value each. A histogram takes continuous numeric observations, divides the number line into intervals, and counts how many observations fall in every interval. One compares categories; the other exposes distribution shape.

10 Correlation Heatmap: Overview of Pairwise Relationships

corr_matrix -> one colored cell for every variable pair



Matrix -> image -> labels -> color scale

heatmap

```
columns = ["heart_rate", "oxygen_level", "temperature"]
corr_matrix = dataframe[columns].corr()
fig, ax = plt.subplots()
image = ax.imshow(corr_matrix, vmin=-1, vmax=1)
ax.set_xticks(range(len(columns)))
ax.set_yticks(range(len(columns)))
ax.set_xticklabels(columns)
ax.set_yticklabels(columns)
fig.colorbar(image)
```

imshow encodes matrix cells as color. Fixing the scale to **[-1, 1]** makes separate correlation plots comparable. A colorbar translates color back into numbers, while ticks and tick labels replace anonymous positions with variable names.

• Template 6 - correlation heatmap

Reusable correlation overview

corr

```
def plot_corr_heatmap(dataframe, columns, output_path):
    corr_matrix = dataframe[columns].corr()
    fig, ax = plt.subplots()
    image = ax.imshow(corr_matrix, vmin=-1, vmax=1)
    ax.set_xticks(range(len(columns))); ax.set_yticks(range(len(columns)))
    ax.set_xticklabels(columns); ax.set_yticklabels(columns)
    fig.colorbar(image)
    fig.savefig(output_path)
    plt.close(fig)
```

Incorrect

Why it fails

Correct

`ax.imshow(corr_matrix)` without `vmin/vmax`

The color range changes with the data, so two heatmaps may look similar while representing different strengths.

`imshow(corr_matrix, vmin=-1, vmax=1)`

Q. When is a heatmap better than the numeric correlation table?

A table is better for exact values. A heatmap is faster for finding direction and the strongest or weakest variable pairs, especially beyond four or five variables. In real analysis, use the heatmap to locate patterns and the table to confirm the exact numbers.

Common Error Summary

Incorrect	Why it fails	Correct
Draw lines without label, then call <code>legend()</code>	Unlabeled lines do not appear in the legend.	Add <code>label=</code> to each line.
Forget <code>plt.close(fig)</code>	Repeated figures accumulate in memory and may leak into later output.	Close after every <code>savefig</code> .
Pass <code>label=mask</code> to <code>scatter</code>	A label names a whole plotted object; it does not color individual points.	Split with the mask and call <code>scatter</code> twice.
<code>ax.boxplot(df["col"])</code>	Only one box is produced; groups are not compared.	Build a list of arrays with a list comprehension.
<code>interpolate()</code> independent rows	Interpolation assumes meaningful order between neighboring observations.	<code>fillna(median)</code>
Heatmap without <code>vmin/vmax</code>	The automatic scale prevents reliable comparison across figures.	<code>imshow(matrix, vmin=-1, vmax=1)</code>

FAQ

Q. What is the exact division of labor between `fig` and `ax`?

`fig` is the complete canvas: it controls saving, size, and multi-panel layout. `ax` is one drawing region: it owns lines, points, bars, titles, and axis labels. Draw and decorate with `ax`; save and close with `fig`.

Q. Why does a scatter plot switch color automatically?

Matplotlib advances a built-in color cycle for each plotting call. That behavior supports the standard grouped-scatter pattern: split the data and call `scatter` separately for each group.

Q. What is the difference between a histogram and a bar chart?

A bar chart gives one named category one value. A histogram automatically divides continuous values into numeric intervals and counts observations in each interval. Bars compare categories; histograms show a distribution.

Q. How do I choose an imputation method in a Capstone problem?

Ask whether neighboring rows are meaningfully ordered. Independent people or transactions usually need a global statistic such as median. Repeated measurements from the same subject or sensor may justify interpolation, forward fill, or backward fill.

Q. When should I use a heatmap instead of the correlation table?

Use the heatmap to scan many relationships and find the strongest directions quickly. Use the numeric table to verify exact values. They are complementary views, not replacements for one another.

13 Exercises

• Practice problems

1. **E Temperature line:** plot `[36.7, 36.9, 38.2, 37.9, 37.1]` with the title `"Temperature over time"`. Practice the fixed four-step skeleton.
2. **E Raw vs three-point rolling mean:** use `[70, 130, 72, 71, 128, 73, 70]`. Draw the raw values and `rolling(3).mean()` in one figure and add a legend. Practice two `plot` calls, labels, and `legend()`.
3. **M Department boxplot:** departments `[A, A, B, B, B]`, scores `[10, 12, 30, 28, 35]`. Practice a list comprehension that separates arrays before `boxplot`.
4. **M Choose a missing-value method:** one patient's heart rate is sampled hourly and the middle two readings are missing. Choose `fillna(median)` or `interpolate()` and explain why. Expected: interpolation, because this is one subject measured continuously; nearby values contain time-local information.
5. **M Correlation heatmap:** use `heart_rate`, `oxygen_level`, and `steps`. Fix the color range to `[-1, 1]`, add labels, and add a colorbar. Practice `corr()` + `imshow` + explicit scaling.

14 Review Order and Chapter Checklist

• Must master

`fig, ax = plt.subplots()` -> draw -> decorate -> save -> close

Several lines: repeated `ax.plot()` + label + `legend()`

Series bar: `ax.bar(series.index, series.values)`

Grouped scatter: split with a mask, then draw each group

Choose imputation by whether observations form an ordered sequence

• Recommended practice

`plt.subplots(rows, cols) + axes[i] + tight_layout()`

`ax.hist(values, bins=n)` for one distribution

`ax.boxplot(groups, tick_labels=...)` with a list of arrays

`imshow(corr_matrix, vmin=-1, vmax=1) + colorbar + labels`

Move old algorithms across containers: Z-score / `groupby` / `fillna`

• Next steps

Seaborn for higher-level statistical graphics

More complex multi-panel layouts

Plotly for interactive charts

Automated visual reports in data pipelines

Move from seeing relationships to modeling them

• Chapter checklist

- Every chart uses the fixed create -> draw -> decorate -> save -> close skeleton.
- A Series bar chart uses its index for x and values for height.
- `plt.subplots(2, 1)` returns an axes array addressed by index.
- `imshow` heatmaps use fixed `vmin/vmax`, labels, and a colorbar.
- The Z-score formula transfers directly after a Series becomes a NumPy array.
- Every line that belongs in a legend has its own `label=`.
- A grouped scatter plot splits the data before separate scatter calls.
- A boxplot receives a list of group-specific arrays, not one unsplit column.
- Missing-value strategy begins with independent records vs a continuous signal.
- Every saved figure is followed by `plt.close(fig)`.

After every exercise: you have completed the full NumPy -> Pandas -> Matplotlib -> Capstone data-processing chain: raw arrays become structured tables, then visual evidence, then integrated judgment. The next step is to move from *seeing relationships* to *training a model to predict them* with classic scikit-learn machine learning.